

The Pricing Signal Problem

A prototype-build challenge for candidates applying to AI Product Engineer, Central Tech

TIME ALLOWED	FORMAT	TOOLS	SUBMIT VIA
3–4 hours	Working prototype	Your choice — vibe coding welcome	GitHub link + Loom walkthrough

BACKGROUND

Opptra manages pricing for 28+ international brands across Amazon India, Noon (UAE), Flipkart, and Lazada. Every brand has a target margin floor, a competitive position it's trying to hold, and a marketplace algorithm that rewards the Buy Box — the featured 'Add to Cart' placement that drives 80%+ of sales.

The pricing team today works mostly in spreadsheets. They check competitor prices manually, make repricing decisions by gut feel, and have no way to know — in real time — whether they're winning or losing the Buy Box across SKUs. When a competitor drops price aggressively overnight, they find out the next morning from a drop in orders.

The core tension: Pricing too high → lose the Buy Box → sales drop. Pricing too low → win the Buy Box → margin erodes. The sweet spot changes every few hours. No one at Opptra currently has a tool that shows them where they stand right now — and what to do about it.

YOUR STAKEHOLDER

Imagine you've just had this conversation with Ranjit, who leads Category Operations:

Ranjit K.

Category Operations
Lead

"Every morning I open three browser tabs — Amazon, Noon, our internal sheet — and manually compare prices for our top SKUs. If a competitor has undercut us, I have to WhatsApp the pricing manager, they approve a change, someone updates it. That's a 3-hour lag minimum. I need something that shows me the problem and suggests a fix — not something that just shows me more data."

That's your brief. He's not asking for a dashboard. He's asking for something that removes the 3-hour lag and gives him a decision, not a spreadsheet.

SAMPLE DATA — WORK WITH THIS

The following represents a snapshot of 8 SKUs across two marketplaces at 9am today. This is the kind of data your tool should be able to process and reason over.

SKU	Brand	Our Price	Competitor	Buy Box	Margin Floor	Last Changed
SKU-001	Natura Casa	Rs.1,299	Rs.1,199 (-100)	Lost	Rs.1,050	3 days ago
SKU-002	Natura Casa	Rs.849	Rs.860	Won	Rs.720	Today
SKU-003	LivSpace Pro	Rs.2,499	Rs.2,199 (-300)	Lost	Rs.1,800	6 days ago
SKU-004	LivSpace Pro	Rs.599	Rs.610	Won	Rs.480	2 days ago
SKU-005	Artisan Home	Rs.3,799	Rs.3,750 (-49)	Lost	Rs.3,200	1 day ago
SKU-006	Artisan Home	Rs.1,150	Rs.1,390	Won	Rs.900	Today
SKU-007	Nordic Basics	Rs.449	Rs.399 (-50)	Lost	Rs.420	5 days ago
SKU-008	Nordic Basics	Rs.2,199	Rs.2,100 (-99)	Lost	Rs.1,750	4 days ago

Note: SKU-007 is a special case — the competitor's price (Rs.399) is below Opotra's margin floor (Rs.420). Your tool should handle this edge case explicitly.

WHAT YOU NEED TO BUILD

Build a working prototype that Ranjit can open in a browser and use in under 60 seconds. It should take the data above as input and give him decisions as output.

1 Ingest & parse the pricing data

Accept the SKU data above — either hardcoded, as a CSV upload, or via a paste interface. Parse it into a structured format your UI and AI layer can reason over.

Hint: Don't overthink the input. A working prototype with hardcoded data beats a broken file uploader.

2 Surface the signal — not the noise

Show Ranjit which SKUs need attention right now: (a) Lost Buy Box items where a price drop is possible within margin, (b) Won Buy Box items with significant headroom (potential to raise price), (c) SKUs where the competitor is below our margin floor — flag these separately, no action possible.

Hint: Ranjit said he doesn't want more data. Prioritise. A triage view works better than a full table.

3 Generate a pricing recommendation with AI

For each actionable SKU, use an LLM to generate a specific recommendation: what price to set, why, and what tradeoff it makes. The output should be a sentence a pricing manager can act on. Example: "Set SKU-001 to Rs.1,189 — Rs.10 below competitor, Rs.139 above margin floor. Recovers Buy Box at 8.4% margin." That is good output. "You should consider lowering the price while being mindful of margins" is not.

Hint: The LLM must be constrained by the margin floor. A recommendation to price below floor is a failure.

4 One-click action simulation

For each recommendation, add an 'Apply' button. It doesn't need to connect to a real marketplace API — a state change that marks the SKU as 'Repriced' and shows a confirmation is enough. The point is that the UX is action-oriented, not read-only.

Hint: This is about the interaction model, not real API integration. Simulate it convincingly.

WHAT TO SUBMIT

REQUIRED GitHub repo link Public or accessible repo. README should explain how to run locally in under 3 steps.	REQUIRED Loom walkthrough (5 min max) Show the prototype working. Walk through your reasoning — what you built, what you cut, what you'd do next.
OPTIONAL One-paragraph design note What was the hardest product decision you made? What would you change with another 4 hours?	OPTIONAL Edge case handling note How did you handle SKU-007 (competitor below margin floor)? What other edge cases did you identify?

HOW WE'LL EVALUATE THIS

We're not grading on polish. We're grading on judgment — does this person understand the problem well enough to make the right tradeoffs under time pressure?

30%	Problem understanding Does the prototype reflect what Ranjit actually asked for? Did they distinguish between 'shows data' and 'gives decisions'?
30%	AI integration quality Is the LLM doing real work — specific, actionable, constrained by the margin floor — or generating filler text dressed as intelligence?

20%	UX & action orientation Can a non-technical pricing manager use this in 60 seconds without explanation? Is it read-only or does it invite action?
20%	Tradeoff awareness Did they handle SKU-007? In the Loom, do they articulate what they cut and why? We want builders who know what they're choosing not to build.

GROUND RULES

- Use any tools, frameworks, or AI coding assistants you like — Cursor, Claude Code, v0, Bolt, Lovable, all fair game. We're testing judgment, not whether you can write code without help.
- You don't need a backend. A well-built frontend with hardcoded data and a real AI call is a better submission than a half-finished full-stack app.
- Don't build a dashboard. Build a decision tool. These are not the same thing.
- The Loom is as important as the code. We want to hear how you think, not just see what you built.
- If you make a product decision we disagree with, explain why you made it. Good reasoning for a wrong call beats a right call with no reasoning.

SUGGESTED STARTING POINTS

You're free to use anything. These are common starting points for candidates who want a hint:

React + Vite · Next.js · Bolt.new · v0 by Vercel · Lovable · Cursor · OpenAI API · Anthropic Claude API · Tailwind CSS · shadcn/ui